

Aufgabenstellung: Abstrakte Klassen in Java

(Klassen „Krokodil“, „Leistenkrokodil“, „Nilkrokodil“ sowie Arbeiten mit abstrakten Oberklassen und konkreten Unterklassen)

Ziel der Aufgabe

In dieser Aufgabe analysieren und entwickeln Sie ein Java-Programm, das eine Klassenstruktur mit einer **abstrakten Oberklasse** und mehreren **konkreten Unterklassen** modelliert.

Sie lernen dabei,

- wie **abstrakte Klassen** in Java verwendet werden,
- wie gemeinsame Eigenschaften und Methoden in einer Oberklasse zusammengefasst werden,
- wie **abstrakte Methoden** in Unterklassen konkret umgesetzt werden,
- wie sich **gemeinsame und spezialisierte Merkmale** unterscheiden,
- wie ein Testprogramm die Beziehungen zwischen abstrakter Oberklasse und konkreten Unterklassen sichtbar macht.

Der Fokus liegt auf dem Verständnis, dass eine **abstrakte Klasse** als allgemeines Modell dient, von dem keine direkten Objekte erzeugt werden, während Unterklassen dieses Modell konkret ausfüllen.

Fachlicher Hintergrund

In der objektorientierten Programmierung gibt es Klassen, die sehr allgemeine Merkmale beschreiben.

Manchmal soll eine solche Klasse **nicht direkt instanziiert** werden, sondern nur als gemeinsame Grundlage für spezialisierte Klassen dienen.

Dafür verwendet man in Java **abstrakte Klassen**.

Eine abstrakte Klasse kann:

- gemeinsame Attribute enthalten,
- bereits implementierte Methoden besitzen,
- aber auch **abstrakte Methoden** vorgeben, die in Unterklassen umgesetzt werden müssen.

Typische Beziehungen in dieser Aufgabe sind:

- ein Leistenkrokodil ist ein Krokodil,
- ein Nilkrokodil ist ein Krokodil,
- beide besitzen gemeinsame Merkmale,
- beide unterscheiden sich aber in bestimmten Eigenschaften oder Verhaltensweisen.

Damit eignet sich dieses Beispiel gut, um den Unterschied zwischen einer **abstrakten Oberklasse** und **konkreten Unterklassen** zu verdeutlichen.

Vorgegebene Klassenstruktur

Im Package `abstrakt.template` sollen folgende Klassen entwickelt und verwendet werden:

- `Krokodil`
- `Leistenkrokodil`
- `Nilkrokodil`
- `Abstrakt`

Die Klassen haben dabei folgende Rollen:

- `Krokodil` ist die **abstrakte Oberklasse**,
- `Leistenkrokodil` und `Nilkrokodil` sind **konkrete Unterklassen** von `Krokodil`,
- `Abstrakt` enthält die `main`-Methode zum Testen des Programms.

Aufgabe 1: Entwicklung der abstrakten Klasse „Krokodil“

Ziel

Sie entwickeln die Klasse `Krokodil` als **abstrakte Oberklasse** der Klassenstruktur.

Zu entwickelnde Klassenstruktur

Die Klasse `Krokodil` soll gemeinsame Merkmale enthalten, die für alle Krokodile gelten.

Beispielhaft kann die Klasse mindestens enthalten:

```
private String name;
```

sowie passende Methoden, zum Beispiel:

```
public Krokodil(String name)
public String getName()
public abstract String gibArtInfo()
```

Zusätzlich kann die Klasse bereits eine gemeinsame Methode enthalten, zum Beispiel:

```
public String beschreibe()
```

Anforderungen

Funktionale Anforderungen

Entwickeln Sie die Klasse `Krokodil` so, dass:

- jedes Krokodil einen Namen besitzt,
- der Name im Konstruktor gesetzt wird,
- der Name über eine Getter-Methode abgefragt werden kann,
- mindestens eine Methode für alle Krokodile gemeinsam nutzbar ist,
- mindestens eine Methode als **abstrakte Methode** vorgegeben wird.

Technische Anforderungen

Verwenden Sie:

- das Schlüsselwort `abstract`,
- ein Attribut `name` vom Typ `String`,
- einen Konstruktor zur Initialisierung,
- mindestens eine Getter-Methode,
- mindestens eine abstrakte Methode.

Ziel dieser Aufgabe

Sie sollen verstehen,

- welche Aufgabe eine abstrakte Oberklasse hat,
 - warum eine abstrakte Klasse nicht direkt zur Objekterzeugung gedacht ist,
 - wie gemeinsame Eigenschaften zentral zusammengefasst werden.
-

Aufgabe 2: Entwicklung der Klasse „Leistenkrokodil“

Ziel

Sie entwickeln die Klasse `Leistenkrokodil` als konkrete Unterklasse von `Krokodil`.

Zu entwickelnde Klassenstruktur

```
public class Leistenkrokodil extends Krokodil
```

Anforderungen

Funktionale Anforderungen

Entwickeln Sie die Klasse `Leistenkrokodil` so, dass:

- sie alle allgemeinen Eigenschaften der Klasse `Krokodil` übernimmt,

- sie die abstrakte Methode der Oberklasse konkret umsetzt,
- sie leistungsspezifische Eigenschaften oder Beschreibungen zurückgeben kann.

Technische Anforderungen

Verwenden Sie:

- `extends Krokodil`,
- einen Konstruktor,
- `super( . . . )` zur Initialisierung geerbter Attribute,
- mindestens eine konkret umgesetzte abstrakte Methode.

Ziel dieser Aufgabe

Sie sollen verstehen,

- wie eine konkrete Unterklasse aus einer abstrakten Oberklasse entsteht,
- warum abstrakte Methoden in Unterklassen umgesetzt werden müssen,
- wie Spezialisierung durch Vererbung erfolgt.

Aufgabe 3: Entwicklung der Klasse „Nilkrokodil“

Ziel

Sie entwickeln die Klasse `Nilkrokodil` als weitere konkrete Unterklasse von `Krokodil`.

Anforderungen

Funktionale Anforderungen

Entwickeln Sie die Klasse `Nilkrokodil` so, dass:

- sie die Merkmale der Klasse `Krokodil` übernimmt,
- sie die abstrakte Methode der Oberklasse mit eigener Logik umsetzt,
- sie sich in mindestens einem Punkt von `Leistenkrokodil` unterscheidet.

Technische Anforderungen

Verwenden Sie:

- `extends Krokodil`,
- einen Konstruktor mit `super(...)`,
- eine eigene Implementierung der abstrakten Methode.

Ziel dieser Aufgabe

Sie sollen verstehen,

- dass mehrere konkrete Unterklassen aus derselben abstrakten Oberklasse entstehen können,
- wie gemeinsame und unterschiedliche Eigenschaften modelliert werden,
- wie abstrakte Klassen eine Klassenhierarchie strukturieren.

Aufgabe 4: Analyse abstrakter Methoden

Ziel

Sie analysieren, wie abstrakte Methoden in Unterklassen umgesetzt werden.

Anforderungen

Funktionale Anforderungen

Untersuchen und entwickeln Sie die Klassen so, dass:

- eine abstrakte Methode in der Oberklasse vorgegeben wird,
- diese Methode in den Unterklassen unterschiedlich umgesetzt wird,
- bei der Ausgabe deutlich wird, welches Krokodil welche konkrete Ausprägung besitzt.

Technische Anforderungen

Beziehen Sie sich auf:

- abstrakte Methoden,
- konkrete Implementierungen in Unterklassen,
- die Wirkung unterschiedlicher Implementierungen.

Ziel dieser Aufgabe

Sie sollen verstehen,

- was eine abstrakte Methode ist,
- warum Unterklassen diese Methode implementieren müssen,
- wie dadurch Polymorphie vorbereitet wird.

Aufgabe 5: Entwicklung der Testklasse „Abstrakt“

Ziel

Sie entwickeln ein Testprogramm, das die Struktur mit abstrakter Oberklasse praktisch demonstriert.

Zu entwickelnde Klasse

Die Klasse `Abstrakt` soll eine `main`-Methode enthalten.

Anforderungen

Funktionale Anforderungen

Das Testprogramm soll:

1. Objekte der konkreten Klassen erzeugen,
2. gemeinsame Methoden der Oberklasse aufrufen,
3. die konkret umgesetzten Methoden der Unterklassen nutzen,
4. die Unterschiede zwischen abstrakter Oberklasse und konkreten Unterklassen sichtbar machen.

Technische Anforderungen

Verwenden Sie:

- eine `main`-Methode,

- mehrere Objekterzeugungen,
 - Methodenaufrufe auf Objekten verschiedener Klassen,
 - sinnvolle Konsolenausgaben.
-

Ziel dieser Aufgabe

Sie sollen verstehen,

- wie eine Klassenstruktur mit abstrakter Oberklasse getestet wird,
 - wie Objekte verschiedener Unterklassen gemeinsam betrachtet werden können,
 - wie Unterschiede in der Ausgabe sichtbar werden.
-

Aufgabe 6: Analyse der Klassenhierarchie

Ziel

Sie analysieren die Beziehungen zwischen der abstrakten Oberklasse und den konkreten Unterklassen.

Anforderungen

Funktionale Anforderungen

Erklären Sie,

- warum Krokodil als abstrakte Oberklasse geeignet ist,
- warum Leistenkrokodil und Nilkrokodil von Krokodil erben,
- warum die Oberklasse nicht direkt instanziiert werden sollte,
- welche Vorteile diese Modellierung hat.

Technische Anforderungen

Beziehen Sie sich auf:

- abstrakte Oberklasse und konkrete Unterklasse,
- gemeinsame Merkmale,
- Wiederverwendung von Code,
- Spezialisierung von Verhalten.

Ziel dieser Aufgabe

Sie sollen verstehen,

- dass abstrakte Klassen ein allgemeines Modell beschreiben,

- warum gemeinsame Eigenschaften nicht mehrfach programmiert werden müssen,
 - wie ein Modell durch abstrakte Klassen übersichtlich und erweiterbar bleibt.
-

Lernziele

Nach Abschluss dieser Aufgabe können Sie:

- erklären, was eine abstrakte Klasse in Java ist,
 - den Unterschied zwischen abstrakter Oberklasse und konkreter Unterklasse beschreiben,
 - Klassen mit `extends` modellieren,
 - Konstruktoren mit `super( . . . )` verwenden,
 - abstrakte Methoden deklarieren und in Unterklassen implementieren,
 - die Klassenstruktur in einem Testprogramm sichtbar machen.
-

Didaktische Einordnung

Diese Aufgabe erweitert grundlegende Java-Kenntnisse um das Konzept der **abstrakten Klassen**.

Sie knüpft an folgende Themen an:

- Klassen und Objekte
- Attribute und Methoden
- Konstruktoren
- Vererbung
- `extends`
- `super(...)`
- abstrakte Klassen
- abstrakte Methoden
- Oberklassen und Unterklassen
- objektorientierte Modellierung

Die Aufgabe zeigt, dass abstrakte Klassen eine sinnvolle Zwischenstufe zwischen allgemeiner Modellierung und konkreter Umsetzung bilden.

Sie bildet eine wichtige Grundlage für weiterführende Themen wie:

- Polymorphie
- dynamische Bindung

- Interfaces
 - UML-Klassendiagramme
 - objektorientierter Softwareentwurf
-

Reflexions- und Kontrollfragen

Kontrollfragen zu abstrakten Klassen

1. Was ist eine abstrakte Klasse in Java?
 2. Warum kann eine abstrakte Klasse nicht direkt instanziiert werden?
 3. Welche Aufgabe hat eine abstrakte Oberklasse?
 4. Welche Aufgabe hat eine konkrete Unterklasse?
-

Kontrollfragen zur Klassenstruktur

5. Warum ist Krokodil eine sinnvolle abstrakte Oberklasse?
6. Warum erben Leistenkrokodil und Nilkrokodil von Krokodil?
7. Welche Gemeinsamkeiten gehören in die Oberklasse?
8. Welche Unterschiede gehören in die Unterklassen?

Kontrollfragen zur Programmlogik

9. Welche Aufgabe hat `super ( . . . )` im Konstruktor?
 10. Was ist eine abstrakte Methode?
 11. Warum muss eine abstrakte Methode in Unterklassen umgesetzt werden?
 12. Welche Vorteile bietet diese Modellierung?
-